

ENGENHARIA DE COMPUTAÇÃO

INTELIGENCIA ARTIFICIAL

Documento de Governança

GenAcademy

Sumário

1. INTRODUÇÃO E OBJETIVO	16
2 RESUMO DO BACKLOG E CAMADAS.....	17
3. ESTRUTURA DAS CAMADAS DO DATA LAKE (BRONZE, PRATA E OURO)	6
3.1 CAMADA BRONZE - DADOS BRUTOS	6
3.2 CAMADA PRATA - DADOS TRATADOS E GOVERNADOS	6
3.3 CAMADA OURO - DADOS ANALÍTICOS E INTELIGÊNCIA.....	7
3.3.1 <i>Dataset de Custo (FinOps)</i>	8
3.3.2 <i>Dataset de Performance</i>	8
3.3.3 <i>Dataset de Segurança</i>	9
4. MODELO UNIFICADO DE VERSIONAMENTO DO PROJETO.....	25
5. LGPD APLICADA AO PROJETO COM FOCO EM CONTROLE E GOVERNANÇA VIA GIT.....	26

1. Introdução e Objetivo

Este projeto tem como objetivo desenvolver uma solução inteligente para análise e otimização de infraestrutura em ambientes de computação em nuvem, utilizando conceitos modernos de engenharia de dados, inteligência artificial e arquitetura de sistemas escaláveis.

A proposta central consiste na criação de um Agente de Otimização de Infraestrutura para SaaS, capaz de analisar grandes volumes de dados provenientes de logs de serviços em nuvem, identificar padrões de uso e gerar recomendações automáticas relacionadas a custo, performance e segurança.

Para isso, o projeto utiliza um dataset público disponibilizado na plataforma Kaggle, contendo registros de eventos de infraestrutura em nuvem. Esses dados representam o comportamento real de aplicações e usuários, permitindo simular cenários comuns enfrentados por empresas SaaS, como picos de acesso, desperdício de recursos e possíveis riscos de segurança.

A arquitetura da solução é baseada no modelo Medallion (Bronze, Prata e Ouro), amplamente utilizado na área de Data Engineering.

No modelo Medallion:

- A camada **Bronze** armazena os dados brutos, garantindo rastreabilidade
- A camada **Prata** realiza o tratamento, limpeza e adequação à LGPD
- A camada **Ouro** transforma os dados em informações estratégicas prontas para análise e inteligência

Além disso, o sistema incorpora técnicas de Machine Learning para detectar anomalias e prever comportamentos, e utiliza o conceito de Retrieval-Augmented Generation para permitir que um modelo de linguagem responda perguntas com base nos dados reais do sistema.

As principais ferramentas e tecnologias utilizadas incluem:

- **Python:** Linguagem usada no processamento de dados;

- **MinIO:** Como Data Lake (armazenamento em estilo S3);
- **PostgreSQL:** Para metadados;
- **FastAPI:** Para a construção da API;
- **MLflow:** Para gestão de modelos de Machine Learning;
- **VectorDB:** Para suportar o mecanismo de busca semântica (RAG).

O propósito do projeto é demonstrar como dados operacionais podem ser transformados em decisões inteligentes e automatizadas, ajudando empresas SaaS a:

- Reduzir custos com infraestrutura
- Melhorar o desempenho das aplicações
- Detectar possíveis problemas de segurança
- Tomar decisões baseadas em dados

De forma simples, o sistema funciona como um assistente inteligente, que observa o comportamento da aplicação e sugere melhorias automaticamente, agregando valor direto ao negócio.

2 Resumo do backlog e camadas

O backlog do projeto foi estruturado de forma estratégica, organizando as atividades em camadas que refletem a arquitetura completa da solução. Essa divisão não apenas facilita o gerenciamento das entregas ao longo das sprints, mas também garante uma separação clara de responsabilidades dentro do sistema, alinhando o desenvolvimento com boas práticas de engenharia de dados, inteligência artificial e construção de aplicações escaláveis.

A Camada de Dados representa a base de todo o projeto, sendo responsável pela ingestão, armazenamento, tratamento e disponibilização dos dados no formato Medallion (Bronze, Prata e Ouro). Essa camada é fundamental, pois garante a qualidade, a rastreabilidade e a confiabilidade das informações que serão utilizadas nas etapas seguintes. Sem uma estrutura de dados bem definida, as análises e modelos de inteligência artificial perderiam consistência e precisão.

A Camada de IA utiliza os dados preparados para gerar valor analítico. Nessa camada são realizadas atividades como engenharia de features, treinamento e avaliação de modelos de Machine Learning, com foco principalmente na detecção de anomalias e análise de padrões de uso. Essa etapa é essencial para transformar dados históricos em previsões e insights, permitindo que o sistema vá além da análise descritiva e passe a atuar de forma preditiva.

A Camada de Aplicação é responsável por disponibilizar os resultados gerados pelas camadas anteriores para o usuário final. Isso inclui a construção de APIs e dashboards que consomem os dados da camada Ouro e os outputs dos modelos de IA. É nessa camada que o projeto se aproxima do conceito de produto, pois os dados e modelos deixam de ser apenas técnicos e passam a ser apresentados como informações acessíveis e úteis para tomada de decisão.

A Camada de MLOps garante a operacionalização dos modelos de Machine Learning ao longo do tempo. Ela é responsável pelo rastreamento de experimentos, versionamento de modelos e monitoramento de desempenho em produção. Essa camada é fundamental para manter a confiabilidade do sistema, permitindo reproduzir resultados, comparar versões de modelos e garantir que as previsões continuem consistentes à medida que novos dados são incorporados.

Por fim, a Infraestrutura sustenta todas as demais camadas, fornecendo os serviços necessários para execução do sistema. Utilizando tecnologias como Docker e Docker Compose para orquestração de ambientes, MinIO como Data Lake e PostgreSQL para armazenamento de metadados, essa camada garante que o sistema seja escalável, reproduzível e de fácil implantação. Além disso, ela permite simular um ambiente próximo ao de produção, o que é essencial para validar a arquitetura proposta.

Em conjunto, essa organização em camadas permite que o projeto evolua de forma estruturada, garantindo que cada etapa do processamento de dados até a entrega de valor ao usuário seja bem definida, desacoplada e alinhada com práticas modernas de desenvolvimento de sistemas orientados a dados.

3. Estrutura das Camadas do Data Lake (Bronze, Prata e Ouro)

O projeto utiliza o modelo de arquitetura Medallion, que organiza os dados em três camadas principais: Bronze, Prata e Ouro. Essa abordagem permite transformar dados brutos em informações estratégicas de forma estruturada, garantindo qualidade, rastreabilidade e geração de valor para o negócio.

3.1 Camada Bronze - Dados Brutos

A camada Bronze representa o primeiro estágio do Data Lake e é responsável pela ingestão dos dados exatamente como são recebidos. Neste nível, os dados provenientes de logs de serviços da AWS, como CloudTrail e CloudWatch, são armazenados sem qualquer tipo de transformação. O principal objetivo dessa camada é preservar a integridade original das informações, garantindo rastreabilidade, auditoria e a possibilidade de reprocessamento completo do pipeline.

Os dados são armazenados no formato CSV e organizados em uma estrutura particionada por data, como por exemplo “dt=YYYY-MM-DD”, o que facilita a organização e a recuperação das informações. Além disso, são aplicadas boas práticas fundamentais, como versionamento dos dados, imutabilidade dos arquivos e a garantia de que nenhum dado será sobrescrito.

Dessa forma, a camada Bronze funciona como uma cópia fiel dos dados de origem, assegurando que qualquer processamento futuro possa ser realizado com segurança e confiabilidade, sem perda de informação ou alteração do conteúdo original.

3.2 Camada Prata - Dados Tratados e Governados

A camada Prata é responsável por transformar os dados brutos em dados confiáveis e utilizáveis. Nesse estágio, os dados passam por processos de limpeza, padronização e adequação às normas de proteção de dados, tornando-se apropriados para análises e aplicações de inteligência artificial.

Entre as principais transformações realizadas estão a remoção de valores nulos e duplicados, a padronização de colunas e a conversão de tipos de dados, especialmente o

campo de tempo. Também é nessa camada que se aplica a conformidade com a LGPD, por meio da anonimização de dados sensíveis, como a substituição do identificador de usuário por hash e o mascaramento de endereços IP.

Além disso, os dados são enriquecidos com novas informações derivadas, como a hora do evento, o dia da semana e a identificação de finais de semana, o que possibilita análises temporais mais avançadas e melhora o desempenho de modelos de Machine Learning.

Os dados processados são armazenados em formato Parquet, que proporciona maior eficiência de armazenamento e melhor desempenho de leitura. Assim, a camada Prata estabelece uma base sólida, limpa e segura, pronta para suportar análises e alimentar etapas mais avançadas do sistema.

3.3 Camada Ouro - Dados Analíticos e Inteligência

A camada Ouro representa o nível mais avançado do Data Lake, onde os dados são organizados como informações estratégicas voltadas diretamente ao negócio. Nesse estágio, os dados deixam de ser apenas registros técnicos e passam a ser estruturados para apoiar decisões relacionadas a custo, performance e segurança, além de alimentar modelos de Machine Learning e o agente inteligente do sistema.

Diferentemente das camadas anteriores, a Ouro não tem como foco a preparação dos dados, mas sim sua organização final para consumo direto, eliminando a necessidade de transformações adicionais. Para isso, os dados são organizados por domínio de negócio, permitindo que cada conjunto responda a perguntas específicas de forma rápida e eficiente.

Todos os datasets da camada Ouro são armazenados em formato Parquet e estruturados para alta performance de leitura, permitindo seu uso direto em dashboards, APIs e sistemas de inteligência sem necessidade de processamento adicional.

Dessa forma, a camada Gold cumpre o objetivo central do projeto: transformar dados técnicos em informações organizadas, interpretáveis e prontas para apoiar decisões estratégicas de forma automatizada.

No contexto deste projeto, a camada Ouro gera principalmente três datasets analíticos principais: custo, performance e segurança.

3.3.1 Dataset de Custo (FinOps)

O dataset de custo tem como finalidade traduzir o uso da infraestrutura em impacto financeiro, sendo um componente essencial para práticas de FinOps e otimização de recursos.

A coluna timestamp representa o horário consolidado do agrupamento e serve como referência principal para análises temporais de custo. Em paralelo, a coluna eventTime registra o primeiro evento real observado dentro daquele agrupamento, permitindo rastreabilidade e auditoria mais detalhada.

As colunas date e hour permitem segmentar os dados por dia e hora, facilitando a identificação de padrões de consumo ao longo do tempo.

O campo service identifica o serviço AWS associado aos eventos, sendo inferido a partir do tipo de ação realizada. Essa informação é fundamental para análises mais granulares, permitindo identificar quais serviços são responsáveis pela maior parte do custo.

A coluna requests representa o volume total de eventos registrados no período, enquanto unique_events indica a diversidade de tipos de evento, fornecendo uma visão mais completa sobre o comportamento de uso.

Por fim, a coluna cost representa o custo estimado do período, calculado com base no volume de requisições e em um custo unitário previamente definido. Essa métrica permite não apenas monitorar gastos, mas também alimentar sistemas de recomendação que sugerem otimizações, como redução de recursos ou ajustes de escalabilidade.

3.3.2 Dataset de Performance

O dataset de performance tem como objetivo analisar o comportamento do sistema em termos de carga, uso e variação ao longo do tempo. Ele permite identificar padrões de

utilização, gargalos operacionais e momentos de sobrecarga, sendo essencial para decisões relacionadas à escalabilidade e otimização da infraestrutura.

Assim como no dataset de segurança, as colunas timestamp, date e hour organizam os dados temporalmente, permitindo análises consistentes ao longo do tempo.

A coluna requests representa o volume total de requisições realizadas no período e constitui o principal indicador de carga do sistema. No entanto, essa métrica isoladamente não é suficiente para caracterizar o comportamento do sistema, sendo necessário analisá-la em conjunto com outras variáveis.

O campo unique_users indica a quantidade de usuários distintos responsáveis pelas requisições naquele intervalo. Essa informação permite diferenciar cenários em que o aumento de carga é causado por muitos usuários simultâneos daqueles em que poucos usuários geram um grande volume de requisições.

De forma complementar, a coluna unique_resources representa a quantidade de recursos distintos acessados, como endpoints ou serviços. Essa métrica ajuda a identificar quais partes do sistema estão sendo mais demandadas, possibilitando a detecção de gargalos específicos.

A coluna requests_growth_pct mede a variação percentual do número de requisições em relação ao período anterior. Esse indicador é essencial para detectar mudanças abruptas no comportamento do sistema, como aumentos repentinos de carga, que podem indicar tanto crescimento legítimo quanto possíveis incidentes.

Já o campo rolling_requests_3 corresponde a uma média móvel das requisições considerando as últimas três horas. Essa abordagem suaviza variações pontuais e permite identificar tendências de crescimento ou queda de forma mais estável.

A coluna is_peak atua como um indicador de pico de uso, sendo geralmente calculada com base em um limiar relativo à média histórica. Quando esse campo é verdadeiro, indica que o sistema está operando em um nível de carga significativamente acima do normal.

Por fim, a coluna `performance_status` sintetiza o estado do sistema em termos interpretáveis, classificando o período como normal, pico ou crescimento abrupto. Esse tipo de abstração é fundamental para permitir que sistemas automatizados e usuários não técnicos compreendam rapidamente a situação operacional.

3.3.3 Dataset de Segurança

O dataset de segurança é responsável por identificar comportamentos fora do padrão e possíveis riscos operacionais a partir da análise dos eventos registrados no sistema. Sua construção considera não apenas a frequência dos eventos, mas também sua raridade, o contexto temporal e a presença de padrões anômalos detectados por modelos de Machine Learning.

A coluna `timestamp` representa o momento consolidado do agrupamento dos eventos, geralmente em nível de hora. Isso significa que todos os eventos ocorridos dentro de um determinado intervalo (por exemplo, entre 08:00 e 08:59) são agregados sob um único registro. Essa padronização permite correlacionar facilmente os dados com outros datasets da camada Gold, como custo e performance.

A coluna `date` complementa essa informação ao registrar apenas a data do agrupamento, permitindo análises em nível diário, enquanto a coluna `hour` explicita a hora do dia, facilitando a identificação de padrões comportamentais, como acessos fora do horário comercial ou atividades concentradas em períodos específicos.

O campo `requests` indica o volume total de eventos registrados naquele intervalo de tempo. Essa métrica é fundamental, pois serve como base para interpretar o restante das variáveis. Um alto número de requisições pode representar tanto um comportamento legítimo (como um pico de uso) quanto um possível ataque ou uso indevido.

A coluna `dominant_event` identifica o tipo de evento mais frequente naquele período. Essa informação é essencial para compreender a natureza da atividade predominante, permitindo diferenciar, por exemplo, entre um pico de leitura de dados e uma sequência repetitiva de ações administrativas potencialmente suspeitas.

Já o campo `rare_events_count` representa a quantidade de eventos considerados raros dentro daquele intervalo. Eventos raros são aqueles que possuem baixa frequência histórica ou que não fazem parte do padrão comum de uso do sistema. Essa métrica ganha ainda mais relevância quando analisada em conjunto com a coluna `rare_events_ratio_pct`, que expressa o percentual de eventos raros em relação ao total de eventos. Um aumento significativo nesse percentual pode indicar um desvio de comportamento, mesmo que o volume total de requisições não seja elevado.

A coluna `anomaly_score` introduz uma camada de inteligência ao dataset, sendo calculada por um modelo de Machine Learning responsável por identificar padrões anômalos. Esse score representa o quão distante o comportamento observado está do padrão esperado, permitindo detectar situações que não seriam facilmente identificadas apenas por regras fixas.

Com base nessas variáveis, o campo `is_unusual_access` atua como um indicador booleano que sinaliza se o período foi considerado suspeito. Essa classificação leva em consideração múltiplos fatores, como o score de anomalia, a presença de eventos raros e o horário da atividade.

A coluna `risk_level` traduz essa análise em uma classificação de risco compreensível, normalmente categorizada como baixo, médio ou alto. Esse campo é especialmente importante por representar uma interpretação direta orientada ao negócio, facilitando a tomada de decisão sem a necessidade de análise técnica aprofundada.

Por fim, a coluna `reason` fornece uma explicação textual para a classificação atribuída, descrevendo os principais fatores que levaram à identificação de risco. Essa explicação é fundamental para o funcionamento do agente inteligente, permitindo que ele apresente respostas interpretáveis e contextualizadas para o usuário final.

4. Modelo Unificado de Versionamento do Projeto

O versionamento neste projeto é tratado como um componente central da arquitetura, sendo responsável por garantir rastreabilidade, reprodutibilidade e controle sobre toda a pipeline de dados e inteligência artificial. Diferente de abordagens fragmentadas, o modelo adotado unifica o versionamento de dados, código e modelos em um único fluxo lógico, permitindo reconstruir qualquer resultado gerado pelo sistema de forma consistente.

A base desse modelo está na integração entre o Data Lake (camadas Bronze, Prata e Ouro), o repositório de código e o ciclo de vida dos modelos de Machine Learning. Cada execução relevante do pipeline gera um conjunto versionado composto por três elementos: a versão dos dados utilizados, a versão do código responsável pelo processamento e a versão do modelo produzido.

No nível de dados, o versionamento é implementado diretamente no Data Lake, utilizando particionamento por data combinado com versionamento semântico. Cada dataset segue um padrão de organização que inclui camada, domínio e data de processamento, além de uma versão explícita (vX.Y.Z). Essa abordagem permite acompanhar a evolução dos dados ao longo do tempo e controlar mudanças estruturais ou correções sem comprometer a consistência dos consumidores.

Para garantir integridade e auditabilidade, cada versão de dataset possui um manifesto associado, que registra informações como arquivos gerados, checksums, schema e metadados de execução. Esse mecanismo é fundamental no contexto do projeto, pois os dados alimentam tanto análises quanto modelos de IA exigindo alto nível de confiabilidade.

No nível de código, o versionamento é realizado por meio de um repositório central em Git, onde cada alteração relevante no pipeline, regras de transformação ou lógica de negócio é registrada por meio de commits versionados. A utilização de branches e Pull Requests garante controle de qualidade e rastreabilidade das mudanças. Cada versão de código utilizada em execução é identificada por um commit específico, que é associado à versão dos dados e modelos gerados.

No nível de modelos, o versionamento considera não apenas o artefato final, mas todo o processo de treinamento. Cada modelo gerado está vinculado à versão do dataset utilizado e ao commit de código correspondente, além de registrar parâmetros e métricas. Isso permite reproduzir experimentos, comparar resultados e evoluir os modelos de forma controlada.

A integração entre essas três dimensões é o principal diferencial do modelo adotado. Ao conectar dados, código e modelos em uma mesma estrutura de versionamento, o projeto garante rastreabilidade completa, permitindo responder de forma precisa a questões como qual versão de dados gerou determinado insight ou qual alteração de código impactou o comportamento do sistema.

5. LGPD Aplicada ao Projeto com Foco em Controle e Governança via Git

A aplicação da LGPD neste projeto é orientada pela natureza dos dados utilizados, que são predominantemente logs técnicos de infraestrutura. Embora esses dados não sejam considerados pessoais diretos, podem conter identificadores indiretos, como endereços IP e identificadores técnicos de usuários, o que exige a adoção de práticas de governança e proteção.

Neste contexto, o principal mecanismo de controle e rastreabilidade adotado no projeto é o uso do Git como ferramenta central de governança das transformações e decisões relacionadas aos dados.

Toda lógica aplicada aos dados — incluindo processos de anonimização, mascaramento e tratamento — é definida em código e versionada no repositório. Isso significa que qualquer alteração relacionada à privacidade ou tratamento de dados pode ser rastreada, auditada e revertida, garantindo transparência e controle.

A anonimização de dados sensíveis ocorre principalmente na camada Silver, onde identificadores como `userId` são transformados em hash e endereços IP são mascarados. Essas regras não são aplicadas diretamente nos dados brutos, mas sim implementadas

como parte do pipeline de transformação, garantindo que a lógica seja versionada e validada.

O uso de Pull Requests para aprovação de mudanças garante que alterações relacionadas à privacidade passem por revisão, evitando a introdução de riscos. Isso é especialmente relevante em cenários onde mudanças de código podem impactar diretamente a exposição de dados.

Além disso, o Git permite manter um histórico completo das decisões tomadas ao longo do projeto, incluindo quando determinada regra de anonimização foi criada, modificada ou removida. Esse histórico funciona como uma trilha de auditoria, essencial para demonstrar conformidade com princípios da LGPD.

Outro ponto importante é a separação entre ambientes e finalidades de uso dos dados. Enquanto a camada Bronze mantém os dados originais, o acesso a dados tratados e anonimizados é controlado e definido por meio das versões do pipeline. Essa abordagem garante que dados sensíveis não sejam expostos indevidamente em etapas analíticas ou de treinamento de modelos.

A segurança é complementada pelo controle de acesso ao repositório, onde apenas usuários autorizados podem realizar alterações. Isso assegura que mudanças críticas sejam controladas e rastreadas.

Por fim, o uso do Git como mecanismo central de governança reforça a transparência e a reprodutibilidade do projeto, permitindo demonstrar não apenas que práticas de proteção de dados foram aplicadas, mas também como e quando foram implementadas.